



VIRTUAL AIR MOUSE

Δ. Παρασκευόπουλος | Σ. Μάκο | Θ. Τραχίλης



Εισαγωγή

Το Virtual-Air-Mouse είναι ένα εικονικό ποντίκι που ελέγχεται αποκλειστικά μέσω κάμερας και χειρονομιών του χεριού.

- Ο χρήστης κινεί το χέρι του μπροστά από την κάμερα και ο κέρσορας ακολουθεί σε πραγματικό χρόνο.
- Τα clicks εκτελούνται με pinch gestures: ο αντίχειρας ακουμπά διαφορετικά δάχτυλα για left, right και middle click.
- Η κάμερα τροφοδοτεί το MediaPipe, που εντοπίζει 21 σημεία στο χέρι, ενώ ένα προσαρμοστικό φίλτρο EMA εξομαλύνει την κίνηση.
- Σχεδιασμένο για διανομές Linux με display server το Wayland.

Προβληματισμοί

Wayland?

Wayland δεν επιτρέπει σε εφαρμογές να μετακινήσουν τον κέρσορα. Οι βιβλιοθήκες αυτοματισμού αποτυγχάνουν.

Λύση: uinput (kernel module) για εικονικές συσκευές εισόδου.

Γιατί REL αντί ABS

ABS θα απαιτούσε:

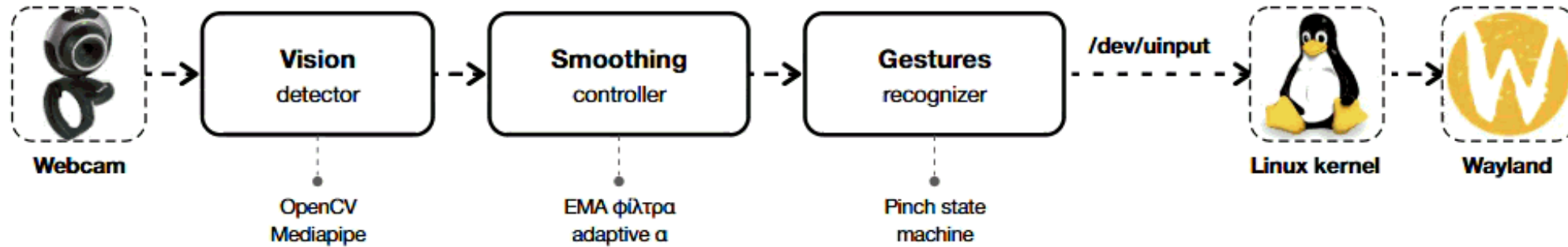
- Stretching κάμερας
- Αντιστοίχιση pixel κάμερας ↔ οθόνης
- Calibration πριν από κάθε χρήση

REL προσέγγιση:

- Πιο απλή υλοποίηση
- Λιγότερο υπολογιστικά κοστοβόρα
- Χωρίς calibration/αντιστοίχιση

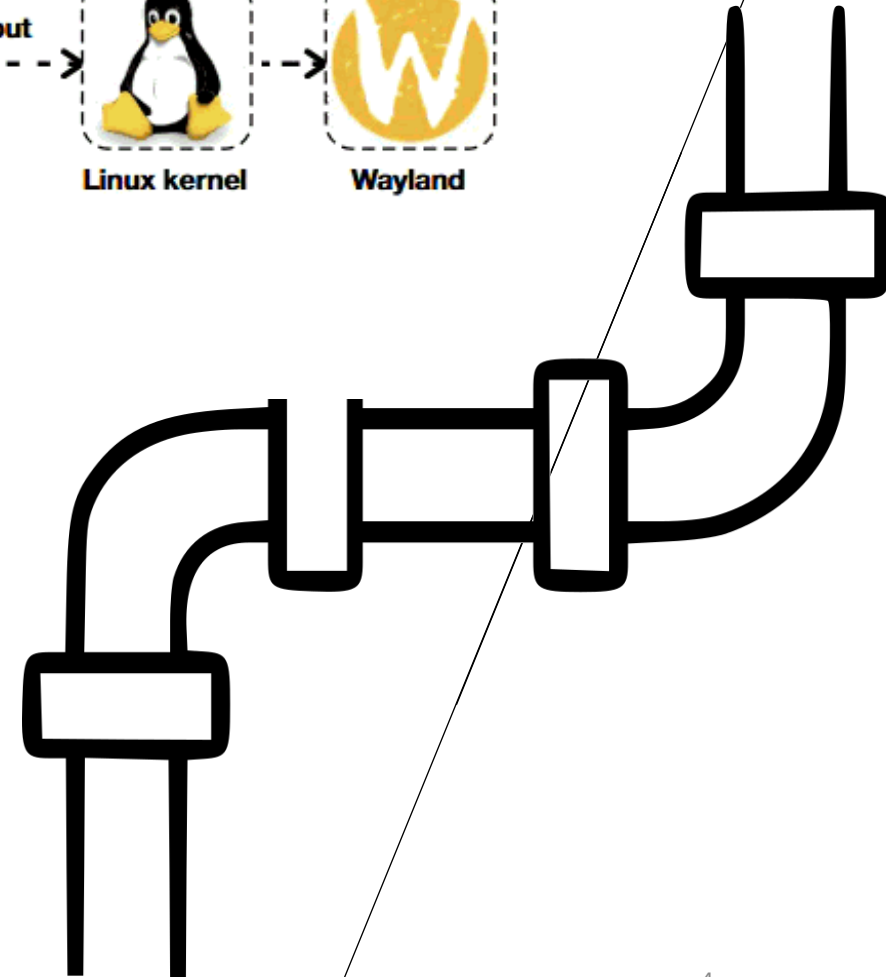


Pipeline



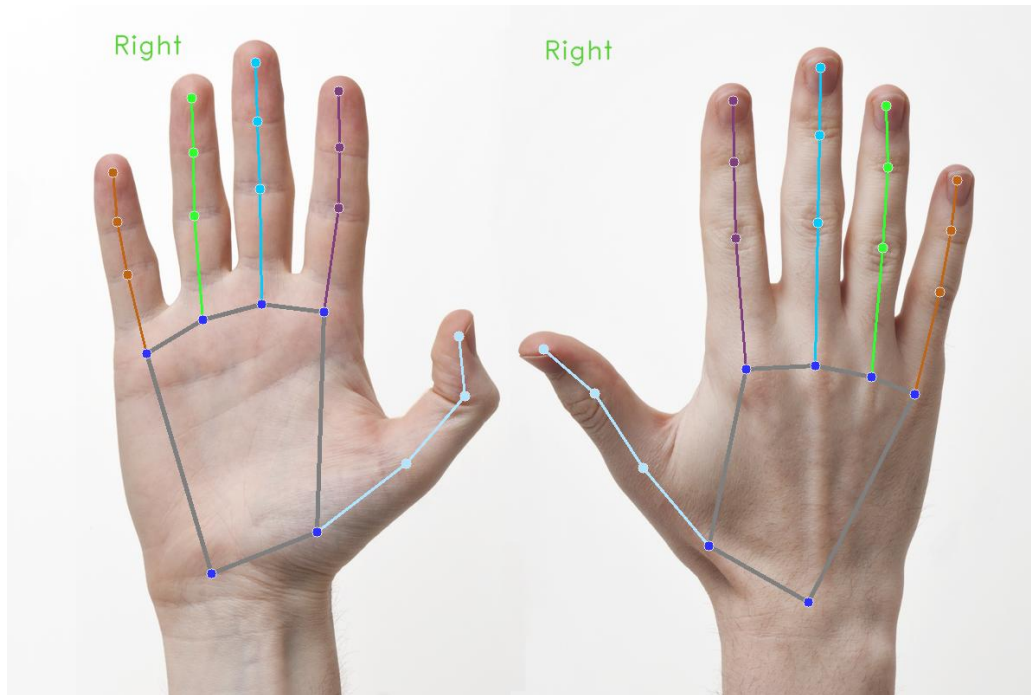
↺ repeats every frame

- Η κάμερα τροφοδοτεί το MediaPipe → 21 landmarks ανά frame
- Το EMA φίλτρο αφαιρεί τον θόρυβο, το adaptive α παγώνει τον κέρσορα όταν το χέρι είναι ακίνητο
- Τα pinch gestures εντοπίζονται από custom μηχανή καταστάσεων
- Ο kernel παραδίδει τα events στο Wayland σαν κανονικό ποντίκι
- Το εικονικό και το φυσικό ποντίκι συνυπάρχουν χωρίς παρεμβολές



MediaPipe: Hand Landmark Detection

- Εντοπίζει 21 σημεία (landmarks) στο χέρι: 4 ανά δάχτυλο + βάση παλάμης



MediaPipe Tasks API

Χρησιμοποιούμε το νεότερο `mediapipe.tasks` API με `.task` model bundles.

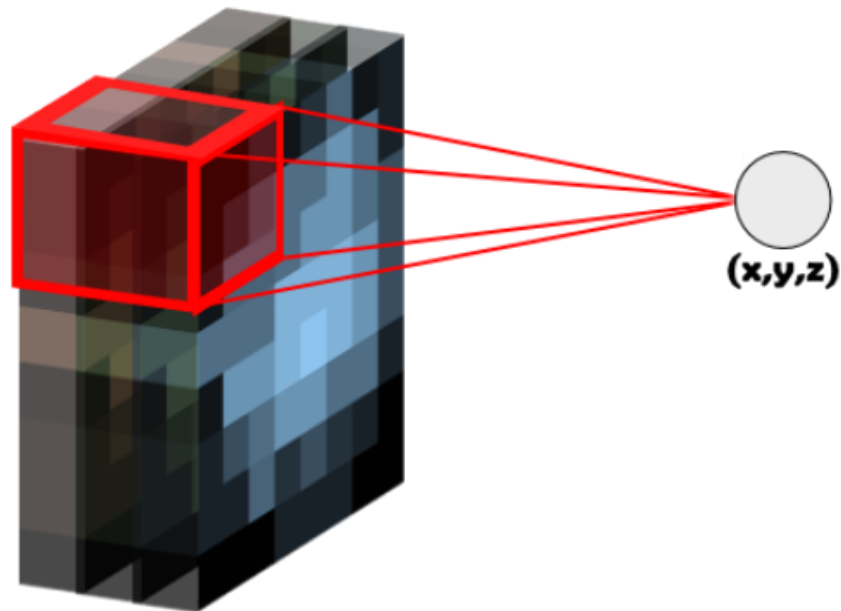
`hand_landmarker.task`

- Model bundle: palm detection + hand landmarks σε ένα αρχείο
- Αντικαθιστά το παλαιότερο `mediapipe.solutions.hands`
- Video mode: tracking μεταξύ frames, μειωμένο latency

Output

- Για κάθε ένα από τα 21 landmarks επιστρέφει μια λίστα συντεταγμένων (x , y , z)
- x , y : κανονικοποιημένα στο $[0,1]$ ως προς πλάτος/ύψος εικόνας
- z : σχετικό βάθος ως προς τον καρπό

Μετατροπή: $\text{Landmark}(X,Y,Z) \rightarrow \text{camera pixel } (X,Y)$



Πώς οι συντεταγμένες μεταφράζονται σε pixel της κάμερας;

Η αντιστοίχιση των συντεταγμένων σε πραγματικά pixels γίνεται πολλαπλασιάζοντας την κανονικοποιημένη τιμή με την αντίστοιχη διάσταση της ανάλυσης.

Τελικά παίρνουμε το ακέραιο μέρος. Αγνοούμε το z , καθώς η κίνηση του κέρσορα γίνεται σε δισδιάστατο χώρο.

Για παράδειγμα

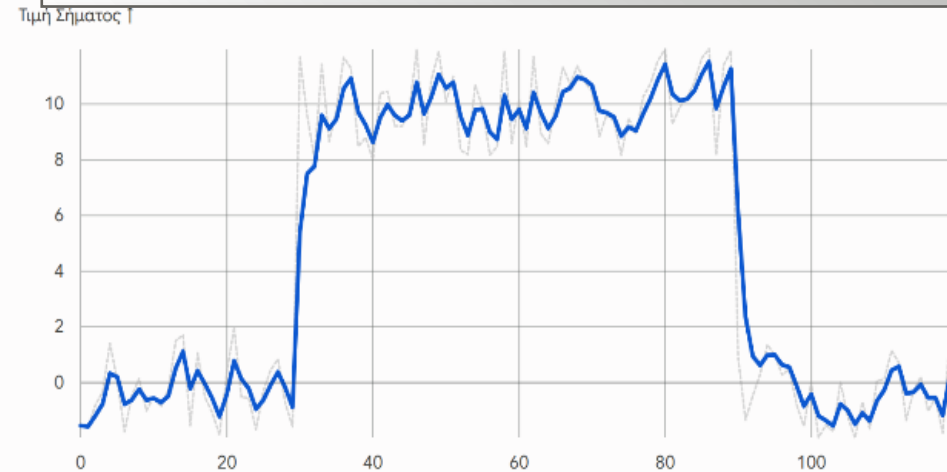
Έστω τις αρχικές συντεταγμένες $x = 0.318$, $y = 0.5$ και διαστάσεις κάμερας **640x480**.

$$\text{row}_x = \text{int}(0.318 \cdot W) = \text{int}(0.318 \cdot 640) = \text{int}(203.52) = 203$$

$$\text{row}_y = \text{int}(0.500 \cdot H) = \text{int}(0.500 \cdot 480) = \text{int}(240.00) = 240$$

Άρα το pixel που αντιστοιχεί είναι το (203,240).

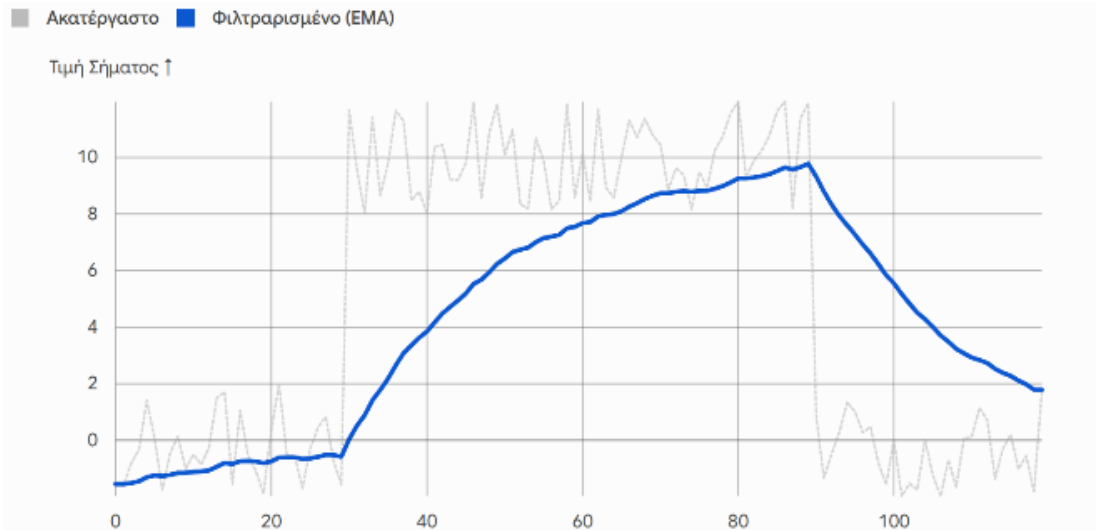
EMA: Exponential Moving Average



Εικόνα 2: Μεγάλο α (π.χ. 0.50) σημαίνει γρήγορη απόκριση, λιγότερη εξομάλυνση.

Αναδρομικό φίλτρο εξομάλυνσης για σταθμισμένο μέσο όρο μεταξύ τρέχουσας μέτρησης και προηγούμενης εξομαλυμένης τιμής.

$$s_t = \alpha \cdot x_t + (1 - \alpha) \cdot s_{t-1}$$



Εικόνα 3: Μικρό α (π.χ. 0.05) σημαίνει αργή απόκριση, μεγάλη εξομάλυνση.

Σύμβολο	Περιγραφή
s_t	Νέα εξομαλυμένη θέση
x_t	Raw θέση από το MediaPipe
α	Συντελεστής EMA
s_{t-1}	Προηγούμενη εξομαλυμένη θέση

STABILITY LOCKING

Το φίλτρο μας είναι **προσαρμοστικό**: το α αλλάζει ανάλογα με την ταχύτητα κίνησης του χεριού. Αυτό συμβαίνει όταν η απόσταση μεταξύ raw και smoothed θέσης είναι **μικρότερη από 15 pixels**, το α μειώνεται στο 10% της κανονικής του τιμής.

$$\text{Έστω } \alpha = \begin{cases} 0.012, & \text{Αν } d < 15 \\ 0.120, & \text{Άλλο} \end{cases}$$

Περίπτωση 1^η: Έστω ότι ο χρήστης κουνάει το χέρι του από το **pixel 500** σε μια νέα θέση, στο **pixel 550**.

$$\begin{aligned} s_t &= 0.12 \cdot x_t + 0.88 \cdot s_{t-1} \Leftrightarrow \\ s_t &= 0.12 \cdot 550 + 0.88 \cdot 500 \Leftrightarrow \\ s_t &= 66 + 440 = 506 \end{aligned}$$

Συνεπώς, η νέα εξομαλυμένη θέση είναι 506.

Περίπτωση 2^η: Έστω ότι ο χρήστης κρατάει το χέρι του ακίνητο, αλλά λόγω θορύβου της κάμερας, το MediaPipe βγάζει λανθασμένα ότι η νέα θέση είναι στο **505 pixel**, με προηγούμενη θέση το **500 pixel**.

$$\begin{aligned} s_t &= 0.012 \cdot x_t + 0.988 \cdot s_{t-1} \Leftrightarrow \\ s_t &= 0.012 \cdot 505 + 0.988 \cdot 500 \Leftrightarrow \\ s_t &= 6.06 + 494 = 500.06 \Leftrightarrow \\ \text{int}(500.06) &= 500 \end{aligned}$$

Συνεπώς, η νέα εξομαλυμένη θέση είναι η ίδια με την προηγούμενη.

ΥΠΟΛΟΓΙΣΜΟΣ ΔΙΑΝΥΣΜΑΤΩΝ ΣΧΕΤΙΚΗΣ ΑΠΟΣΤΑΣΗΣ

1. Σχετική μετατόπιση: 2. Συσσωρευτής κίνησης: 3. Τελική κίνηση & υπόλοιπο:

$$\Delta x_{\text{cam}} = s_{x,t} - s_{x,t-1} \quad A_{x,\text{prev}} = A_{x,\text{new}} \quad dx = \lfloor A_x \rfloor$$
$$A_x = A_{x,\text{prev}} + (\Delta x_{\text{cam}} \cdot S) \quad A_{x,\text{new}} = A_x - dx$$

Παρομοίως για τον άξονα y

- Επειδή το ποντίκι δέχεται μόνο ακέραιες τιμές, η απλή στρογγυλοποίηση θα μηδένιζε τις πολύ αργές κινήσεις (π.χ. $\lfloor 0.8 \rfloor = 0$). Για να λυθεί αυτό, τα δεκαδικά υπόλοιπα προστίθενται σε συσσωρευτές (accumulators).

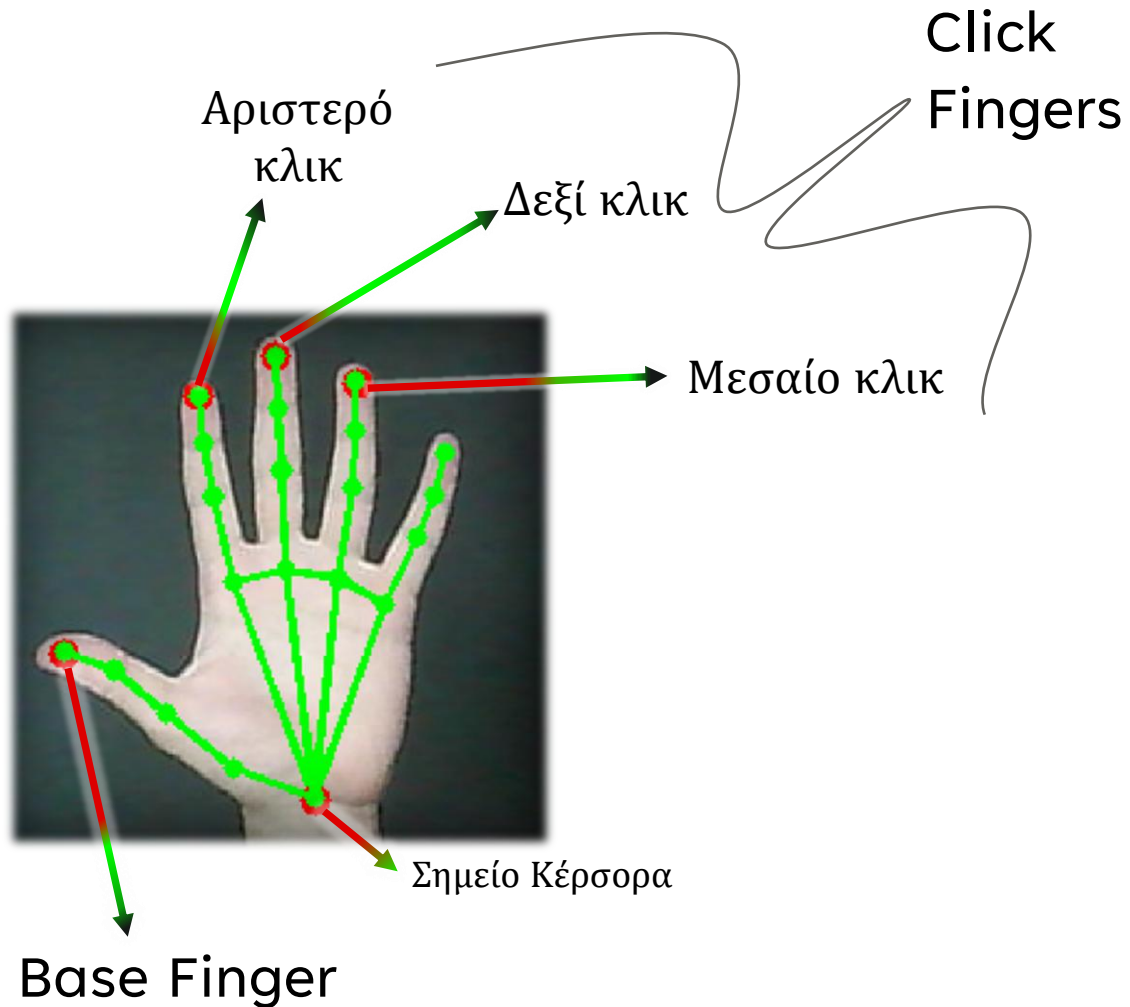
Παράδειγμα (μόνο άξονας x)

Έστω $S = 3.5$, $A_{x,\text{prev}} = 0.4$, $s_{x,t} = 506$, $s_{x,t-1} = 500$

- $\Delta x_{\text{cam}} = 506 - 500 = 6$
- $A_x = 0.4 + (6 \cdot 3.5) = 0.4 + 21 = 21.4$
- $dx = \lfloor 21.4 \rfloor = 21 \rightarrow \text{REL_X} = 21 \text{ στον kernel}$
- $A_{x,\text{new}} = 21.4 - 21 = 0.4 \rightarrow \text{κρατείται για επόμενο frame}$

Δx_{cam}	Καθαρή μετατόπιση σε pixel κάμερας
$s_{x,t}, s_{y,t}$	Τρέχουσα εξομαλυμένη θέση
S	Συντελεστής κλιμάκωσης (sensitivity)
dx	Τελική ακέραια κίνηση $\rightarrow$ kernel
$A_{x,\text{new}}$	Δεκαδικό υπόλοιπο $\rightarrow$ επόμενο frame
$A_{x,\text{prev}}$	- -

Αντιστοίχιση χειρονομιών σε ενέργειες συστήματος



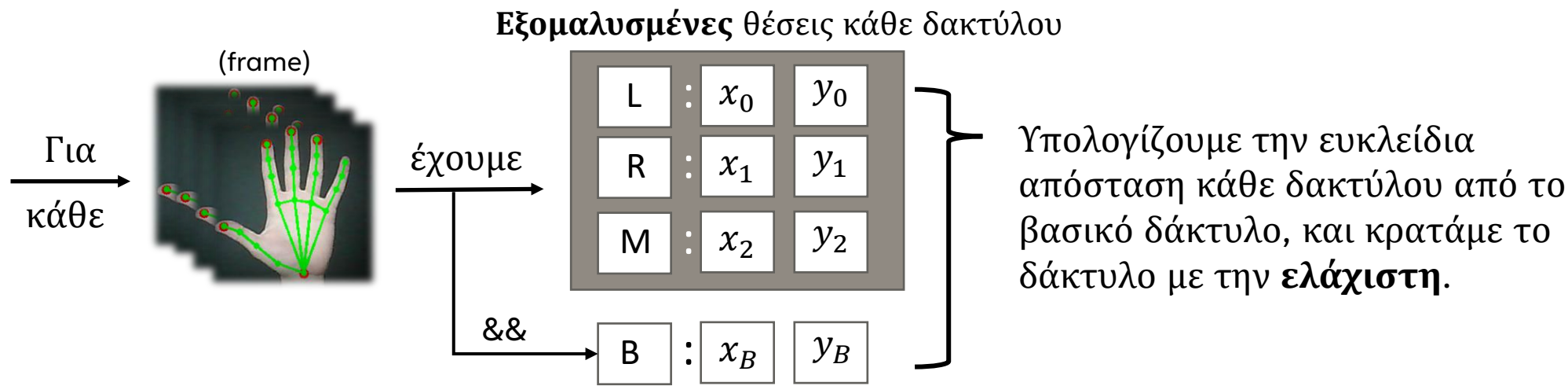
Ο συνδυασμός Base & Click Finger παράγει στιγμιαίο ή παρατεταμένο Click

Η κλασσική χειρονομία κλικ (το απλό πάτημα)



Για να εντοπίσουμε μια χειρονομία λοιπόν χρειαζόμαστε την τωρινή και την προηγούμενη κατάσταση χειρονομίας

Πως βρίσκουμε τις καταστάσεις χειρονομίας?



Ύστερα, με βάση ένα παραμετροποιήσιμο κατώφλι απόστασης, αν έχουμε:

- **Ελάχιστη απόσταση < κατώφλι:**

Στέλνουμε στην μηχανή καταστάσεων την αντίστοιχη κατάσταση χειρονομίας (LEFT, RIGHT, MIDDLE) του δακτύλου με την ελάχιστη ευκλείδια απόσταση

- **Ελάχιστη απόσταση > κατώφλι:**

Στέλνουμε στην μηχανή καταστάσεων την άδεια κατάσταση χειρονομίας (NONE).

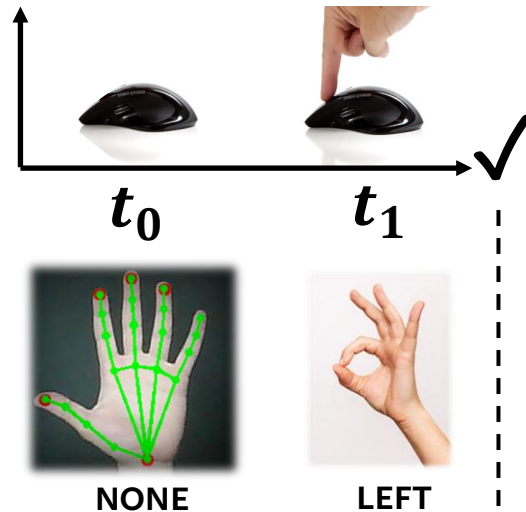
Η μηχανή καταστάσεων:

- διατηρεί την αμέσως προηγούμενη κατάσταση χειρονομίας
- αποθηκεύει την καινούρια που της στείλαμε,

ώστε να αρχίσει να αποφασίζει για το πότε και αν θα γίνει κλικ

Διαχείριση των gesture states από την μηχανή καταστάσεων

Δύο βασικές περιπτώσεις



Σύνδεση Δακτύλων

Έχουμε όταν η κατάσταση χειρονομίας στο t_{new} δεν είναι η άδεια (NONE)

Αν τωρινή κατάσταση $\neq$ προηγούμενης

- Καταγραφή ώρας έναρξης σύνδεσης με αντίχειρα (t_{now})

Αν τωρινή κατάσταση = προηγούμενη (σε επόμενο καρέ)

- Μετράμε διάρκεια σύνδεσης αντίχειρα:

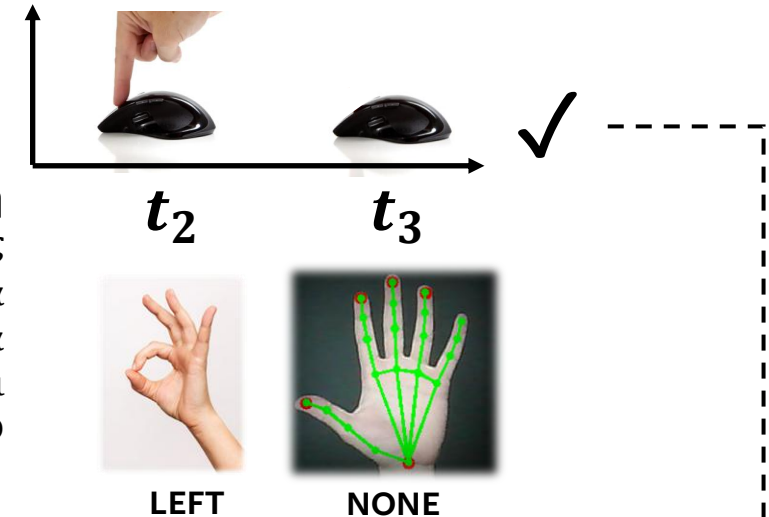
$$t_{now} - t_{εναρξη}$$

- Αν διάρκεια $>$ κατώφλι χρόνου παρατεταμένου πατήματος:

Ενεργοποιούμε παρατεταμένο πάτημα για την ενέργεια του t_1

Αποσύνδεση δακτύλων

Έχουμε όταν η κατάσταση χειρονομίας στο t_{new} είναι η άδεια (NONE) και η παλιά κατάσταση t_{new-1} είναι είτε L, ή R, ή MID



Καταγραφή & Υπολογισμός:

- Χρόνου αποσύνδεσης
- συνολικής Διάρκειας σύνδεσης με αντίχειρα

Αν $\Delta <$ Κατώφλι στιγμιαίου πατήματος:

Αγνοούμε το σήμα (θεωρούμε κίνηση ως εσφαλμένη θορυβώδη ενεργοποίηση)

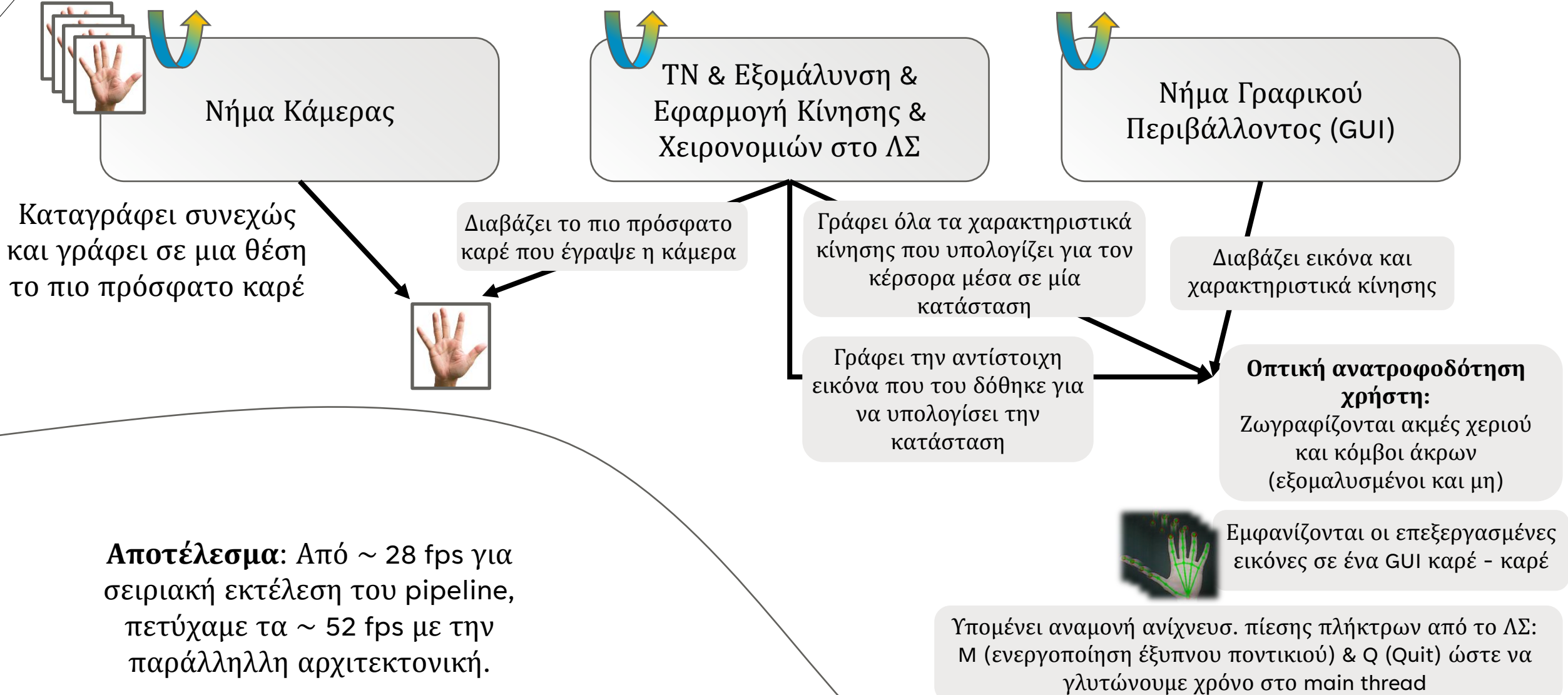
Αν $\Delta >$ Κατώφλι στιγμιαίου πατήματος:

Και $\Delta <$ Κατώφλι παρατεταμ. πατήματος:

- Ενεργοποιούμε το στιγμιαίο κλικ για την χειρονομία του t_{new-1}

Επιπλέον, αν ήταν ενεργοποιημένη η παρατεταμένη λειτουργία αφήνεται αμέσως αντί να γίνει κλικ.

Βελτιστοποίηση Αρχιτεκτονικής για καθημερινή χρήση



ΕΥΧΑΡΙΣΤΟΥΜΕ!

ΑΝΑΦΟΡΕΣ

- [1] Google LLC. 2024. Hand landmarks detection guide. MediaPipe. Retrieved from https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- [2] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214. Retrieved from <https://arxiv.org/abs/2006.10214>
- [3] Linux Kernel Developers. 2023. uinput: User-level input driver. Linux Kernel Documentation. Retrieved from <https://www.kernel.org/doc/html/latest/input/uinput.html>
- [4] Georgi Valkov. 2023. python-evdev: Python bindings to the Linux input handling subsystem. Retrieved from <https://python-evdev.readthedocs.io/>
- [5] OpenCV Team. 2024. Open Source Computer Vision Library. Retrieved from <https://docs.opencv.org>
- [6] The Wayland Project. 2024. Wayland Protocol. Retrieved from <https://wayland.freedesktop.org>
- [7] Charles R. Harris, K. Jarrod Millman, Stefan J. van der Walt, et al. 2020. Array programming with NumPy. Nature 585 (2020), 357–362. DOI: <https://doi.org/10.1038/s41586-020-2649-2>

ΕΙΚΟΝΕΣ

- <https://thumbs.dreamstime.com/b/finger-clicking-computer-mouse-11286732.jpg>
- https://t4.ftcdn.net/jpg/04/69/08/99/360_F_469089985_0fvUF52LCdDH6X6UEdoBKao7V1bVsDs7.jpg
- https://t4.ftcdn.net/jpg/05/54/78/89/360_F_554788951_fLay5C8e9bha4caBTWVJN6rvTD0pEVfE.jpg